



Search

CS 541 Lecture 4

Zining Zhu





From learning to search

Two threads · one course

Overview of this lecture

Lecture 1 previewed two toolkits: **learning from data** and **search over structure**. Today we start the second thread.

- Search underlies routing, planning, puzzle-solving, and (later) game-playing AI
- We build a common vocabulary, then survey five algorithms in order of sophistication
- Same skeleton every time: what does each algorithm expand next, and what does it guarantee?

By the end: implement BFS, DFS, IDS, Dijkstra, and A*, and choose among them.



A running example, used all lecture long

- Every algorithm today will be traced, by hand, on the **same** small weighted graph
- Watch for one repeated punch line: three of the five algorithms **do not** find the cheapest route — only two do
- The difference is not a bug in the first three; it is exactly what “complete” and “optimal” do and do not promise

Keep this slide’s promise in mind — we will cash it in at the end of the lecture.



A brief history, in one slide

- 1959: **Dijkstra's algorithm** — shortest paths in a weighted graph, for a very different problem (network routing), later recognized as uniform-cost search
- 1968: **A*** — Hart, Nilsson, and Raphael at SRI, built for a robot named Shakey that had to plan routes through a room
- Both ideas predate “AI” as a field name, but both are core exhibits in it today
- The pattern repeats across this course: a general algorithm from one field turns out to **be** the AI algorithm for a different problem



Search as a problem-solving framework

States · actions · goals

What is a search problem?

A search problem asks: how do you get from a start state to a goal, given a set of allowed moves?

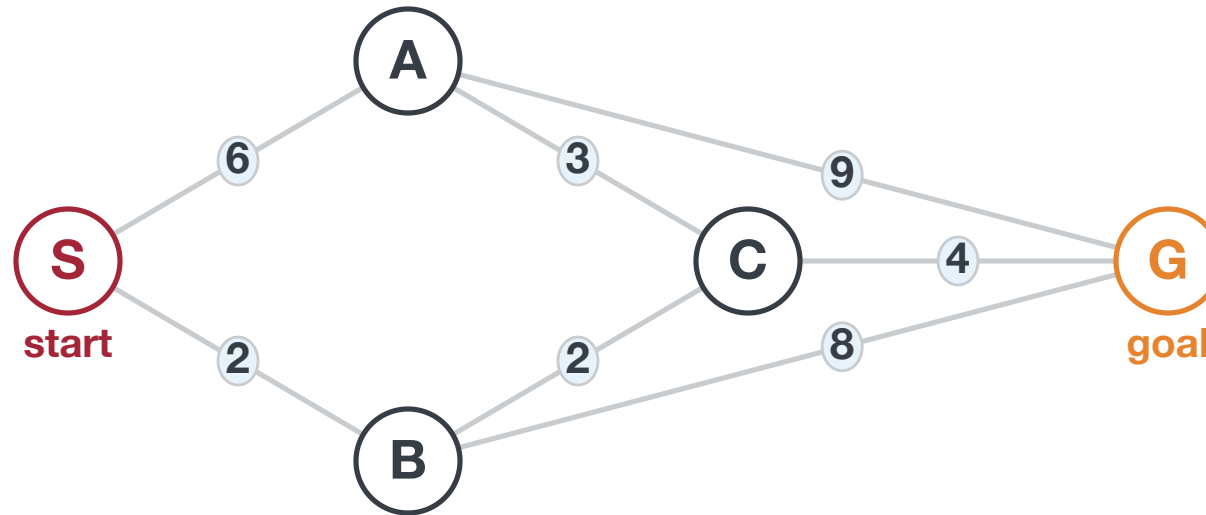
- **State space:** the set of all configurations the world could be in
- **Actions:** the moves available from a given state
- **Transition model:** $\text{result}(s, a)$ — the state you land in after action **a** in state **s**
- **Goal test:** a predicate that says whether a state counts as “done”
- **Path cost:** a sum of step costs along a sequence of actions

A **solution** is a sequence of actions from the start state to a goal state.



The running example: a road-trip map

Five towns, connected by roads with drive times (in minutes). We will trace every algorithm on this exact graph.



edge labels are drive times, in minutes

Start = S. Goal = G. All roads are two-way.

Formalizing the road-trip example

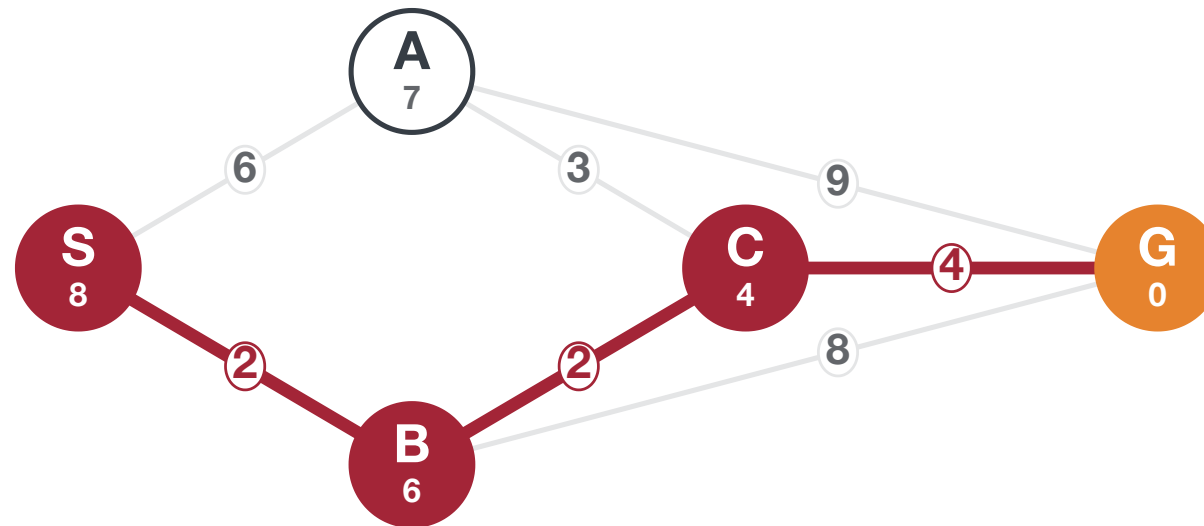
- **State:** which town you are currently in
- **Actions:** drive along any road leaving the current town
- **Transition model:** driving a road ends in the town at its other end
- **Goal test:** is the current town G?
- **Path cost:** sum of drive times along the roads taken

Small enough to trace completely by hand — that is the point. The algorithms behave identically on a graph with a million nodes.



The true cheapest route (so we can grade every algorithm)

Working it out once, in advance, from G backward: G costs 0, C costs 4 via C–G, B costs 6 via B–C–G, A costs 7 via A–C–G, and S costs 8.

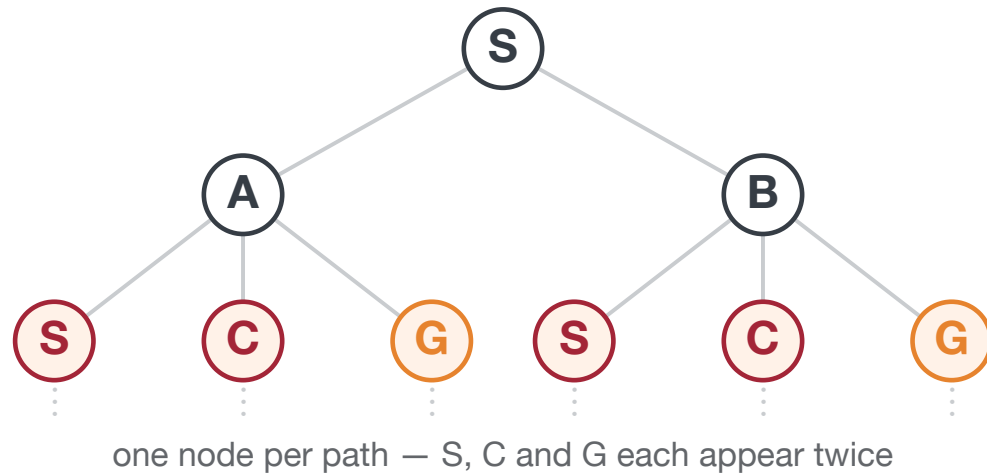


$S \rightarrow B \rightarrow C \rightarrow G = 2 + 2 + 4 = 8$
the small number is that town's cheapest cost to G

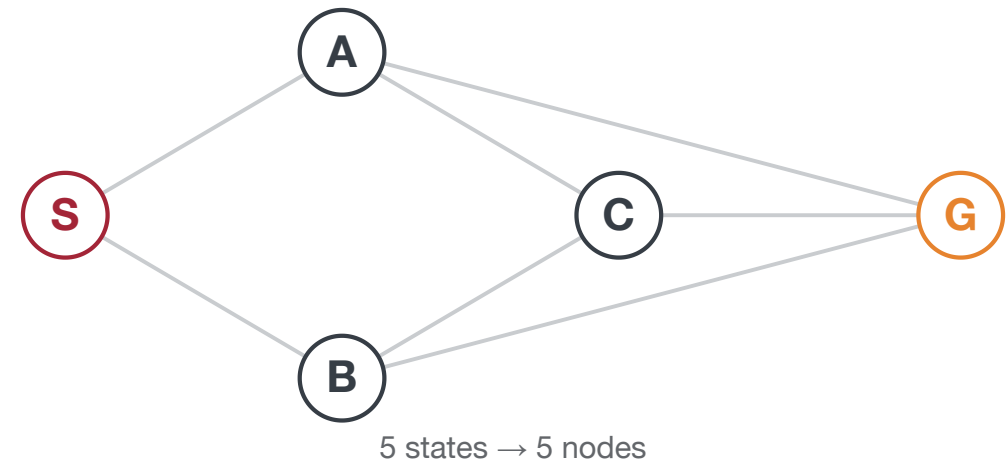
The optimal route from S is **$S \rightarrow B \rightarrow C \rightarrow G$** , **cost 8**. Remember this number.

Search tree vs. search graph

- A **search tree** unfolds the state space along paths, so one state can sit at many nodes
- The **state space graph** underneath has exactly one node per state



Search tree



State space graph

Searching the tree is simpler to reason about but can revisit states wastefully. Today's algorithms differ mainly in **which node they expand next**.

Why repeated states matter here

- Our road-trip graph has a cycle: S–A–C–B–S
- A **tree** search that ignores this can loop: $S \rightarrow A \rightarrow C \rightarrow B \rightarrow S \rightarrow A \rightarrow \dots$ forever
- A **graph** search fixes this with one extra piece of bookkeeping: remember which states were already expanded

Every algorithm we implement in Assignment 3 is a **graph** search, for exactly this reason.



Frontier and explored set

- **Frontier:** the set of nodes generated but not yet expanded
- **Explored / visited set:** nodes already expanded, so we don't repeat work
- **Expanding** a node = generating all of its successors via the transition model
- A node only needs to be added to the frontier if it is not already in the frontier **and** not already in the explored set

This one check is what turns a tree search into a graph search.



The general search algorithm template

1. Initialize frontier with the start state
2. Loop:
 - If frontier is empty, fail
 - Choose and remove a node from the frontier (**this choice is the whole game**)
 - If it is a goal, return the solution
 - Otherwise, expand it and add its unvisited successors to the frontier

Every algorithm below is this loop with a different frontier data structure. The frontier's data structure **is** the algorithm.



Tree-search vs. graph-search pseudocode, side by side

	Tree search	Graph search
Extra state	none	an <i>explored</i> set
Before adding a successor	always add it	add only if not in frontier and not in explored
Risk	can loop forever on a cyclic graph	none, at the cost of remembering visited states
Used in this course	never	always



Check your understanding

- Name the five ingredients of a search problem, using the road-trip example for each
- Why can a tree search loop forever on a graph with a cycle, but a graph search cannot?
- What is the one design choice that distinguishes BFS, DFS, and Dijkstra from each other?

(The answer to the third question is coming up next.)





Uninformed search

Blind to the goal

Breadth-first search (BFS)

Frontier is a **FIFO queue**: expand the shallowest unexpanded node first.

- Explores level by level — all depth- d nodes before any depth- $d+1$ node
- **Complete**: always finds a solution if one exists (in a finite space)
- **Optimal** only when all step costs are equal
- Time and space: $O(b^d)$, where b = branching factor, d = solution depth



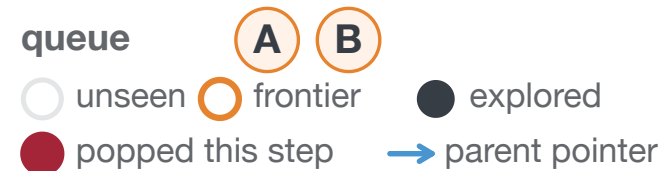
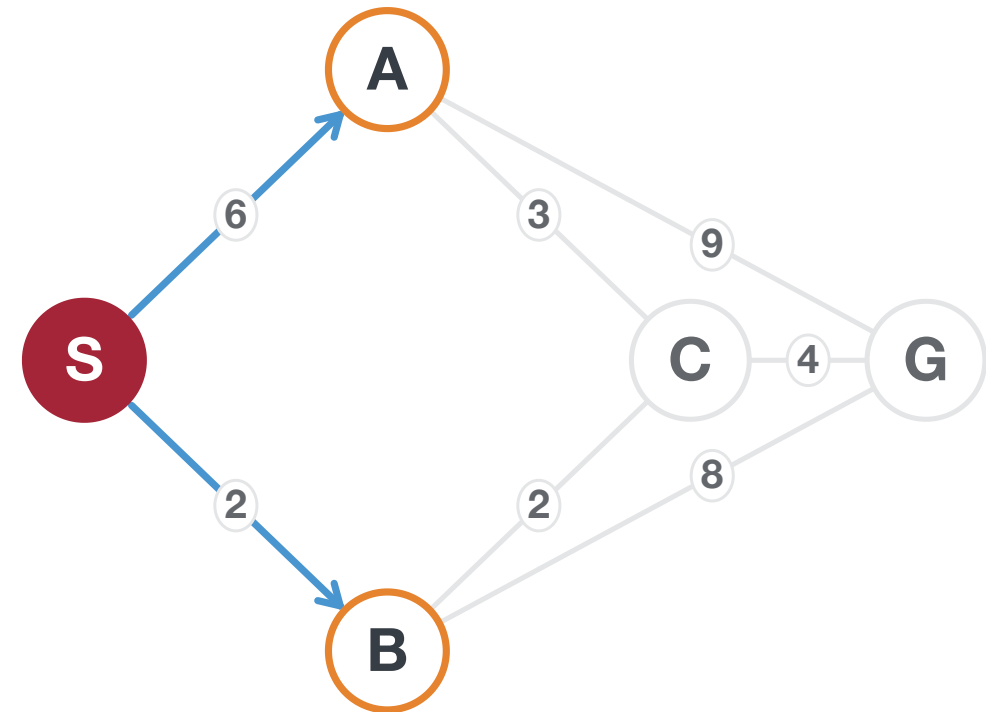
BFS on the road trip: setting up

- Frontier starts as the queue [S]; explored set starts empty
- Successors are generated in alphabetical order at every expansion, so the trace is deterministic
- We track, for every node the first time it enters the frontier, *who* put it there (its parent) — this is what lets us reconstruct the path at the end



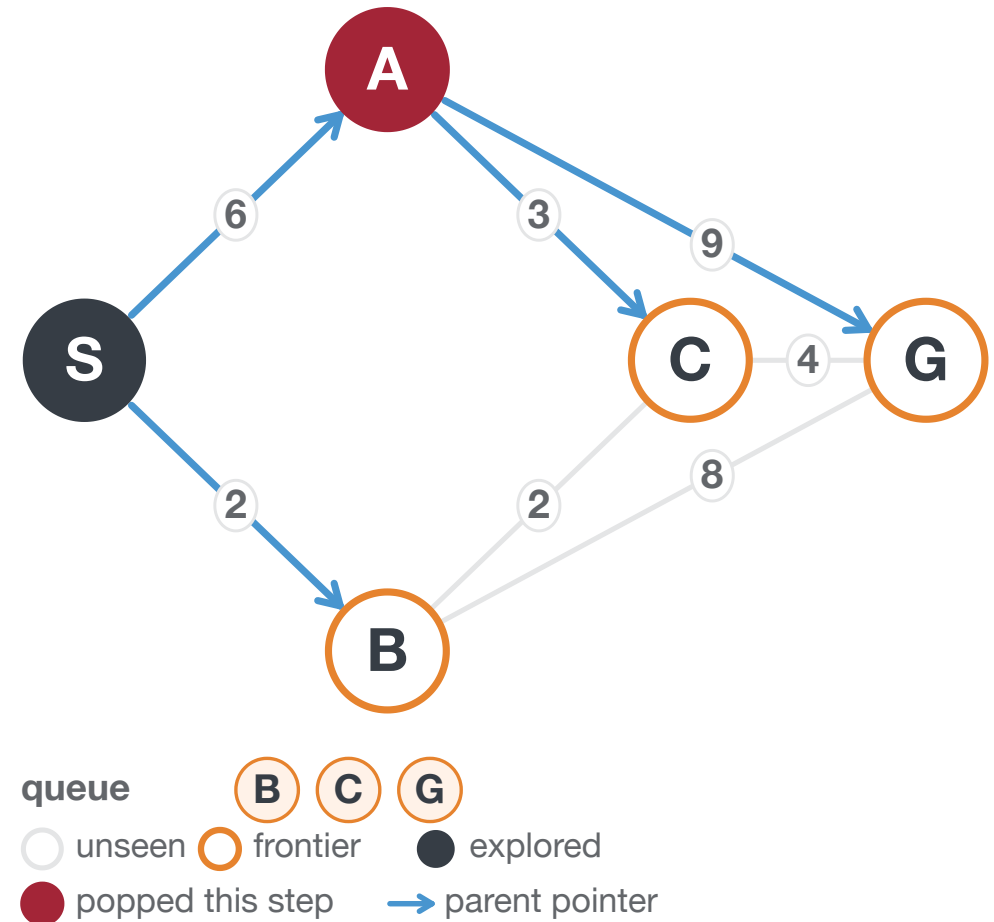
BFS trace, step 1

- Pop **S** from the front of the queue. Mark S explored.
- S is not the goal.
- Expand S: successors A, B (alphabetical). Neither is in the frontier or explored, so both are added.
- Queue afterward: [A, B]



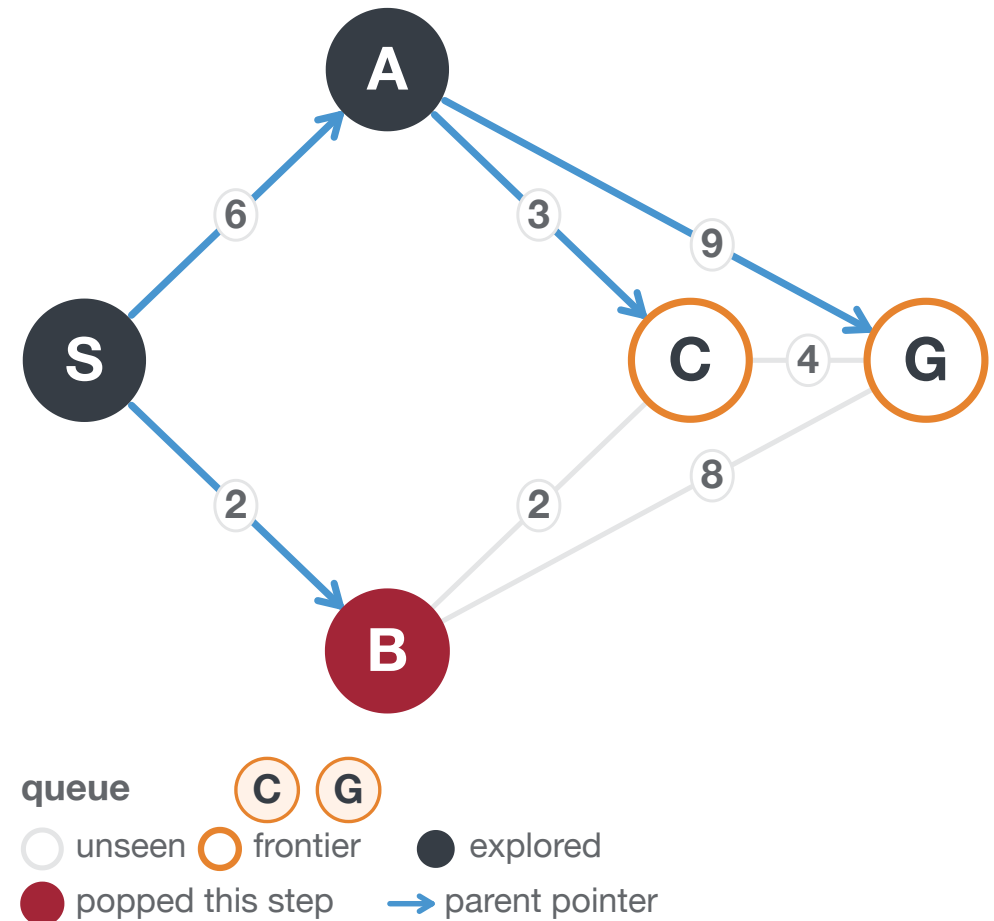
BFS trace, step 2

- Pop **A**. Mark A explored (parent of A is S).
- A is not the goal.
- Expand A: successors C, G, S. S is already explored — skip it. Add C and G.
- Queue afterward: [B, C, G] — and note: **G's parent is now A**.



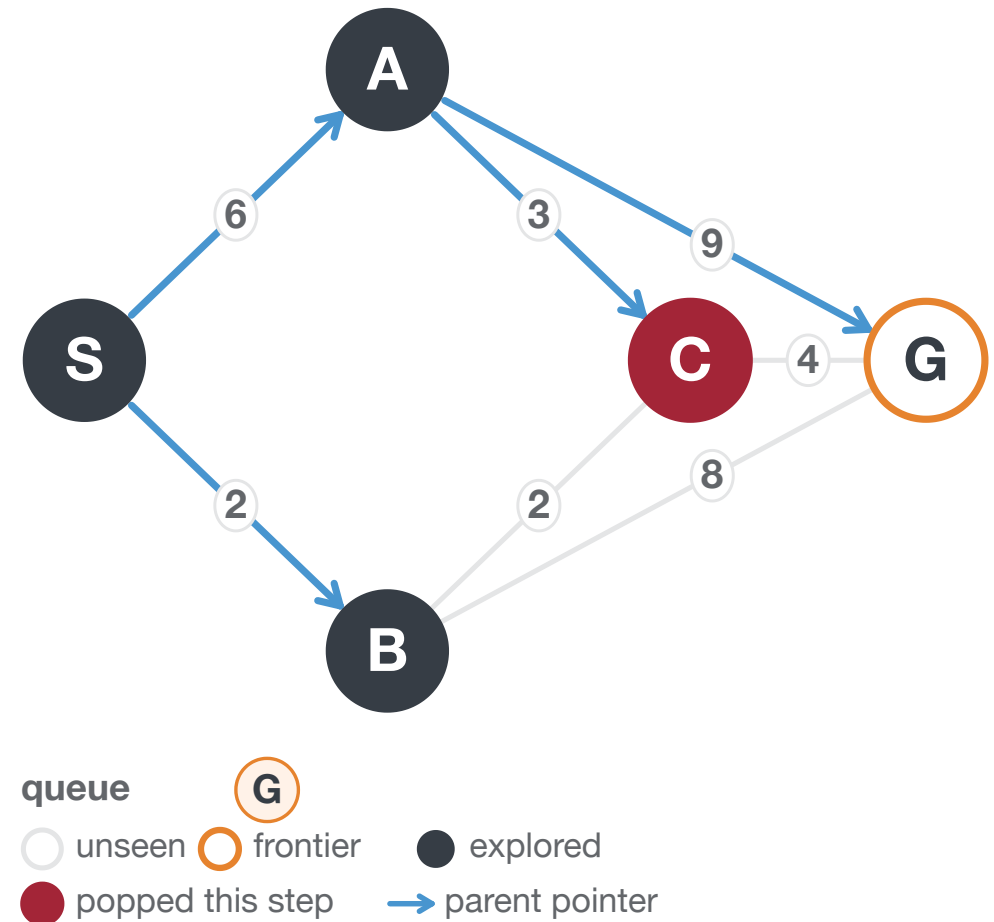
BFS trace, step 3

- Pop **B**. Mark B explored (parent of B is S).
- B is not the goal.
- Expand B: successors C, G, S. C and G are already in the frontier — skip both (no duplicate parents). S is explored — skip.
- Queue afterward: [C, G] — unchanged by this expansion.



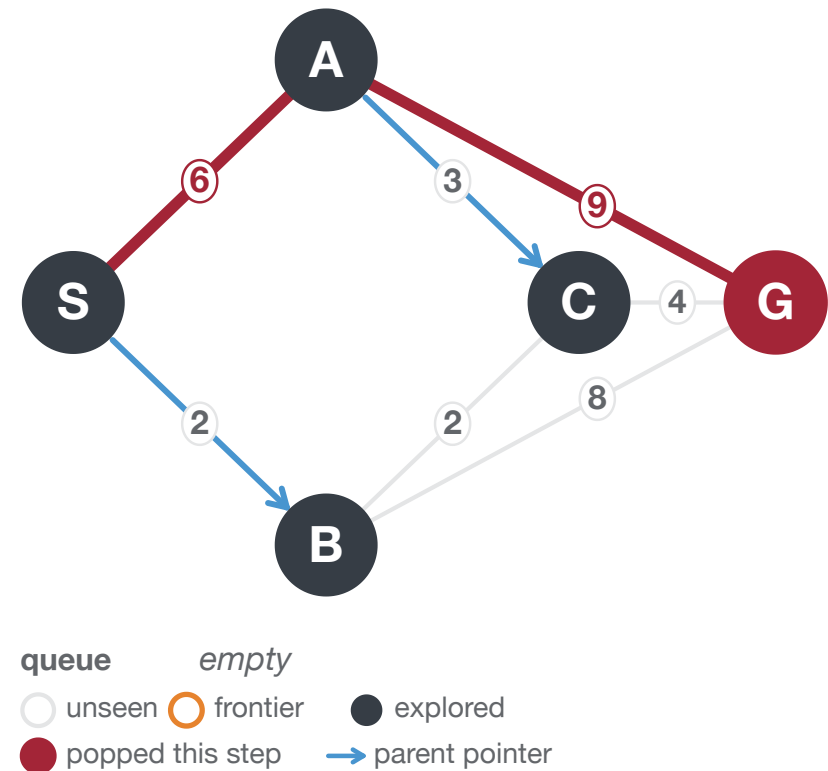
BFS trace, step 4

- Pop **C**. Mark C explored (parent of C is A).
- C is not the goal.
- Expand C: successors A, B, G. A and B are explored — skip. G is already in the frontier — skip.
- Queue afterward: [G]



BFS trace, step 5 — done

- Pop **G**. It is the goal!
- Reconstruct the path by following parent pointers backward: $G \leftarrow A \leftarrow S$.
- **BFS's answer: $S \rightarrow A \rightarrow G$, cost $6 + 9 = 15$.**



Nodes expanded, in order: S, A, B, C, G — all five. Cost found: 15. Recall the true optimum is 8.

BFS: why it missed the optimal route

- BFS chose the **shallowest** unexpanded node, not the **cheapest**
- S–A–G is 2 hops; S–B–C–G is 3 hops — BFS prefers fewer hops, full stop
- It never compares 15 against 8, because it never looks at edge weights at all

This is not a bug — it is exactly what “optimal only under unit step costs” means.



BFS: why space is the bottleneck

- Must store the entire frontier — at depth d , that is $O(b^d)$ nodes
- Memory, not time, is usually what kills BFS in practice
- Good fit when: solutions are shallow, and all actions cost the same (e.g. unweighted graphs, puzzle solving by move count)



Depth-first search (DFS)

Frontier is a **LIFO stack** (or recursion): expand the deepest unexpanded node first.

- Space: $O(bm)$ — only one path plus siblings at each level, $m = \text{max depth}$
- **Not complete** in infinite or cyclic state spaces (can loop forever down one branch)
- **Not optimal** — the first goal found need not be the cheapest



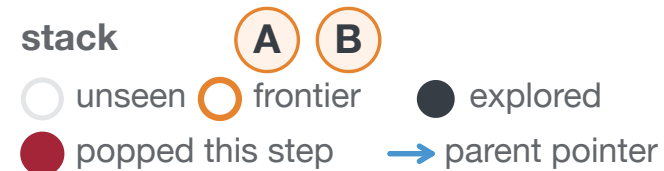
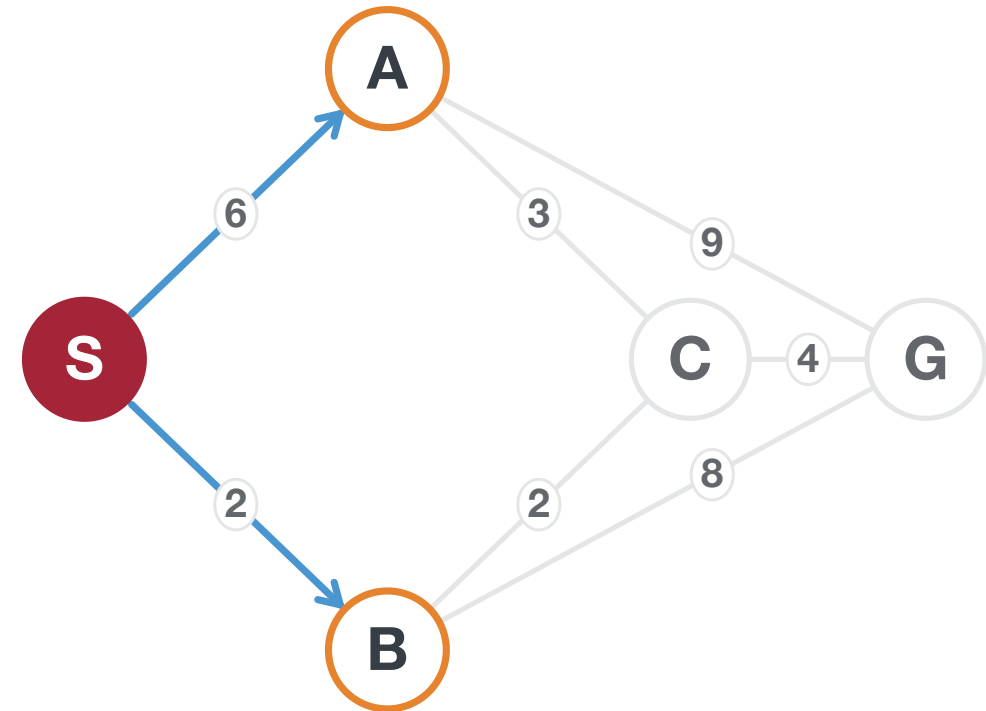
A stack subtlety worth getting right

- If successors are generated alphabetically (A before B) and pushed in that order, the **stack** pops B first — last pushed, first popped
- To explore “A first” the way the alphabetical order suggests, push successors in **reverse** order (B, then A), so A lands on top
- This trace uses that convention throughout — it is exactly the kind of off-by-one detail that bites in Assignment 3



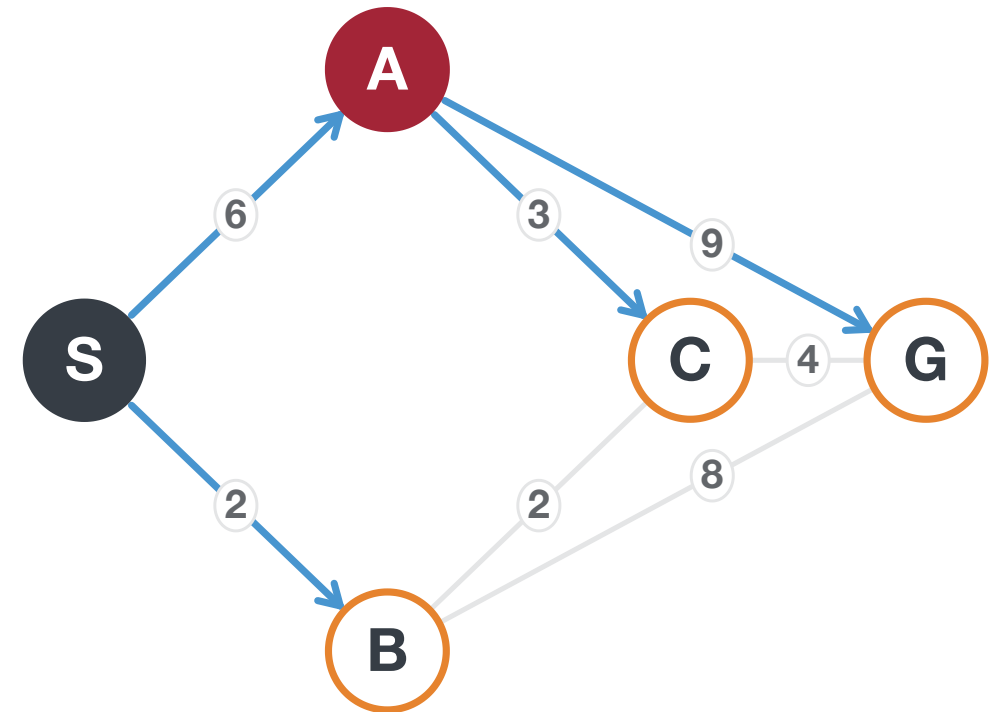
DFS trace, step 1

- Stack starts as [S].
- Pop **S**. Mark explored. Not the goal.
- Successors A, B; push B then A (A ends on top). Stack: [A, B] (top listed first)



DFS trace, step 2

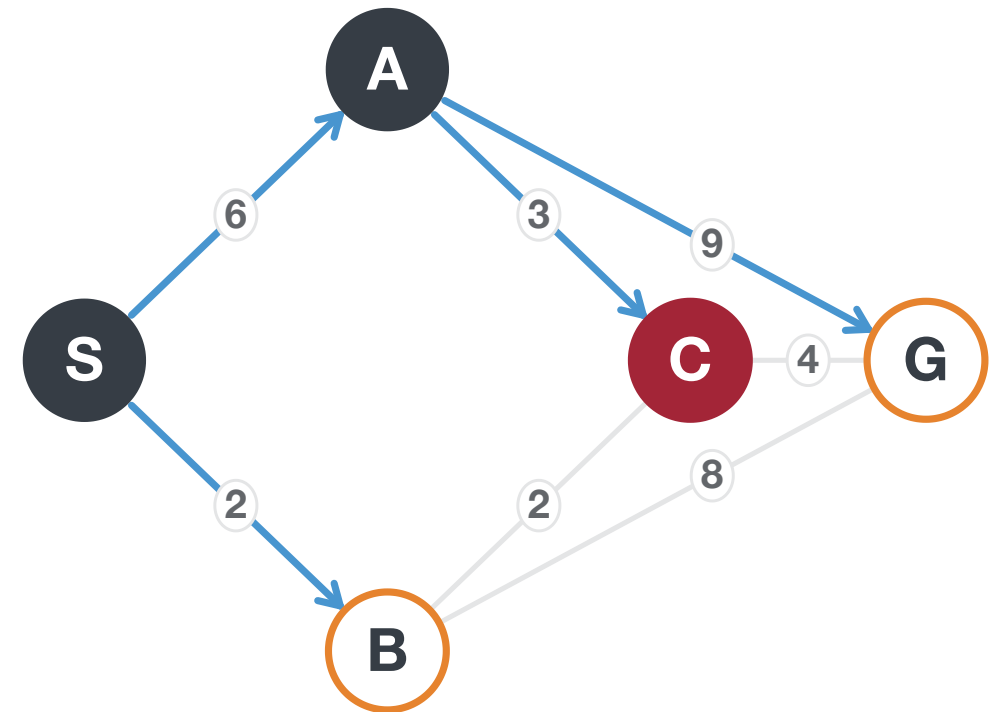
- Pop **A**. Mark explored (parent of A is S). Not the goal.
- Successors of A, excluding explored S: C, G. Push G then C (C ends on top).
- Stack: [C, G, B] — note **G's parent is A**.



stack C G B
unseen frontier explored
popped this step parent pointer

DFS trace, step 3

- Pop **C**. Mark explored (parent of C is A). Not the goal.
- Successors of C, excluding explored A: B, G. Both are *already in the frontier* (stack) — a graph search skips re-adding them.
- Stack unchanged after removing C from the top: [G, B]

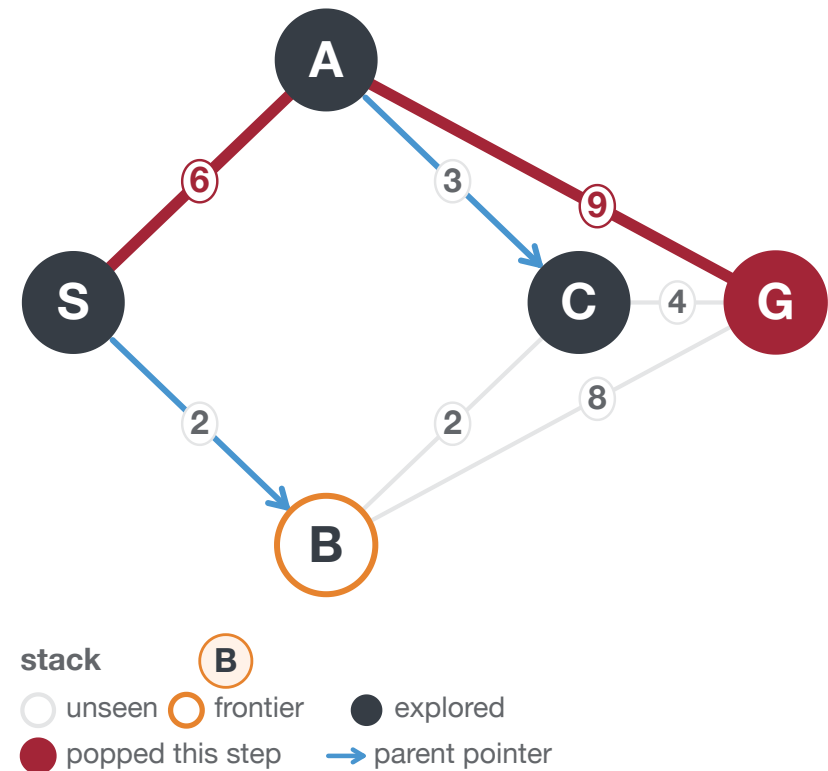


stack [G, B]

○ unseen ○ frontier ● explored
● popped this step → parent pointer

DFS trace, step 4 — done

- Pop **G**. It is the goal!
- Parent pointers: $G \leftarrow A \leftarrow S$ (G's parent was fixed at step 2 and never revisited).
- **DFS's answer: $S \rightarrow A \rightarrow G$, cost 15** — the same suboptimal route BFS found, and town B was never even expanded.



Nodes expanded, in order: S, A, C, G — only four. DFS can be *faster to a goal* while still being *wrong about which goal is cheapest*.

BFS vs. DFS on this graph, side by side

	BFS	DFS
Nodes expanded	S, A, B, C, G	S, A, C, G
Nodes touched	all 5	only 4 (B never visited)
Route found	S–A–G	S–A–G
Cost found	15	15
Optimal?	no	no

Same wrong answer, different amounts of work to get there. Neither looked at a single edge weight.



DFS: the incompleteness trap

- Without a depth bound or a visited-set check, DFS can wander down an infinite path and never backtrack
- **Depth-limited search:** cut off expansion at depth ℓ — trades completeness for a memory guarantee
- Sets up the next algorithm: what if we don't know the right limit ℓ in advance?



Iterative deepening search (IDS)

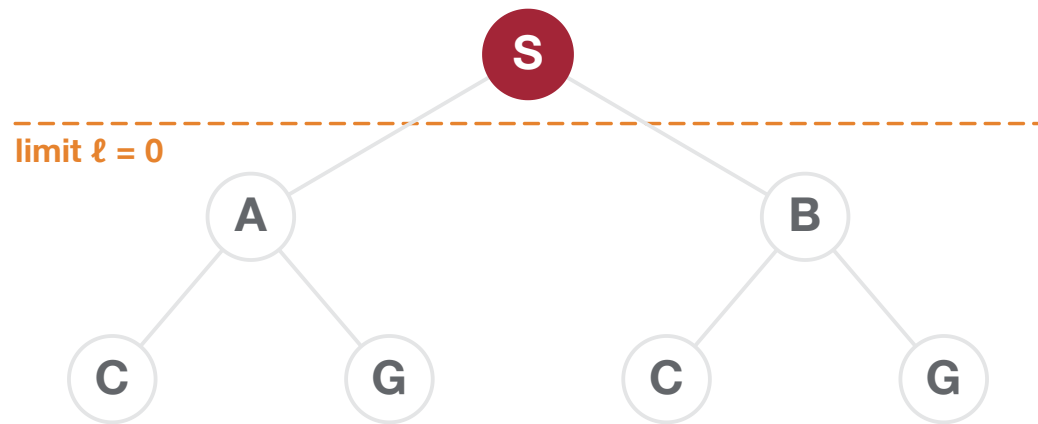
Run depth-limited search with limit $\ell = 0, 1, 2, \dots$ until a goal is found.

- Combines BFS's completeness/optimalty (under uniform costs) with DFS's $O(bd)$ space
- Seems wasteful — re-expands shallow nodes every iteration — but the cost is dominated by the last iteration
- Time: $O(b^d)$, same order as BFS; the repeated work is a constant-factor overhead

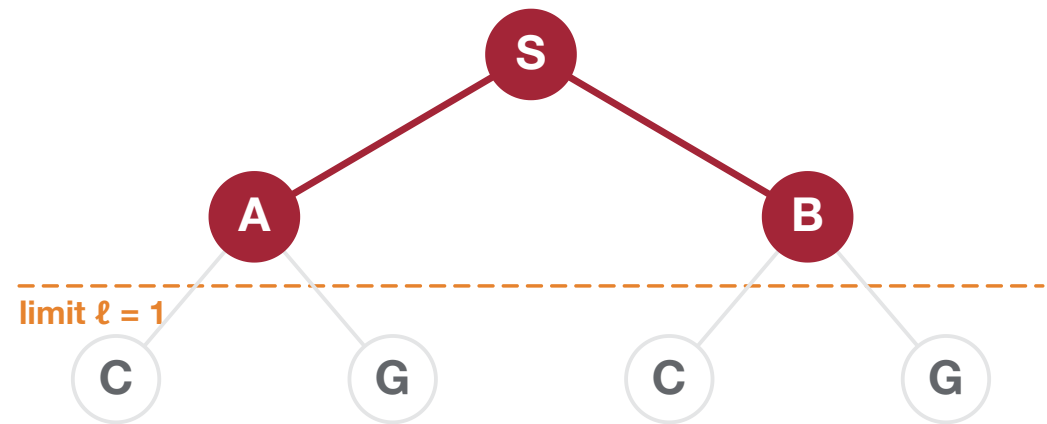


IDS trace: $\ell = 0$ and $\ell = 1$

- $\ell = 0$: only S itself is within the limit. S is not the goal. Fail, increase limit.
- $\ell = 1$: expand S to depth 1 — reach A and B. Neither is the goal (G is not among them). Fail, increase limit.



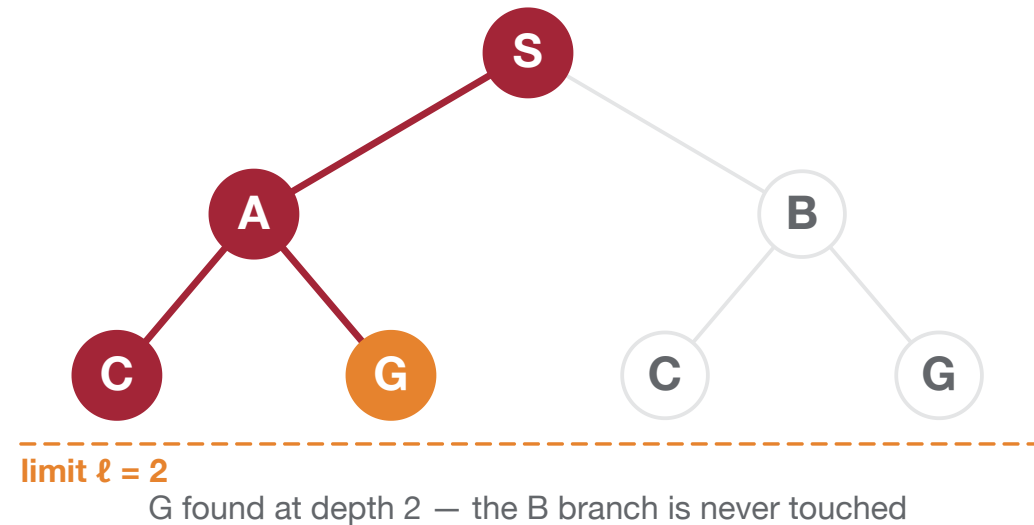
S is not the goal — fail, raise the limit



A and B are not the goal — fail, raise the limit

IDS trace: $\ell = 2$ — succeeds

- Depth-limited DFS to depth 2, same push convention as before: from S, visit A first (depth 1), then A's children C and G (depth 2)
- **G appears at depth 2**, reached via $S \rightarrow A \rightarrow G$ — goal test passes immediately
- Depth-limited search never even reaches the $S \rightarrow B$ branch at this limit, because A's subtree already produced a goal



IDS's answer: $S \rightarrow A \rightarrow G$, cost 15 — the third algorithm in a row to report the “wrong” route, and for the same underlying reason.

IDS: why the repeated work is only a constant factor

- Nodes at depth d are generated once; nodes at depth $d-1$ are regenerated twice; depth $d-2$ three times; and so on
- The sum is dominated by the last, deepest layer — total work is still $O(b^d)$, not $O(d \times b^d)$
- Traded: a little re-computation, for a large memory saving over BFS

IDS is the default uninformed choice for large, unknown-depth spaces.



Uninformed search: a quick comparison

Algorithm	Complete?	Optimal?	Time	Space
BFS	yes	yes (unit cost)	$O(b^d)$	$O(b^d)$
DFS	no (infinite/cyclic)	no	$O(b^m)$	$O(bm)$
IDS	yes	yes (unit cost)	$O(b^d)$	$O(bd)$



Where uninformed search shows up in practice

- **BFS**: shortest number of hops in a social network (“degrees of separation”), unweighted shortest paths, level-order tree traversal
- **DFS**: topological sort, cycle detection, maze generation and solving, exploring a file system tree
- **IDS**: chess endgame tablebase construction and other settings where the solution depth is unknown in advance and memory is the binding constraint

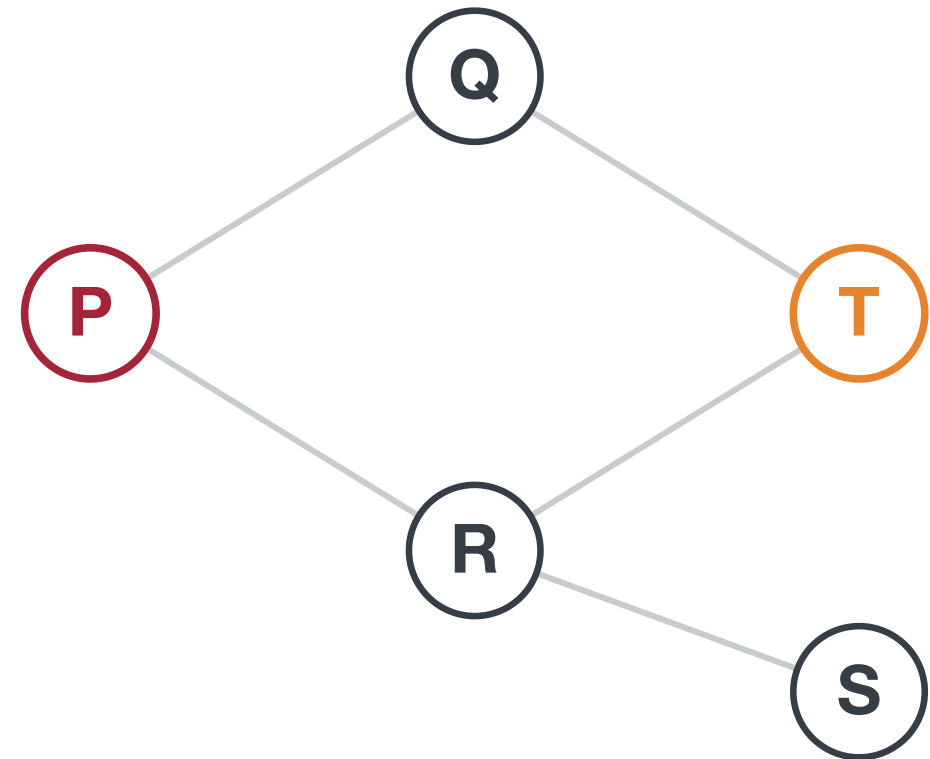
None of these applications involve edge weights — exactly the setting where “fewest steps” and “cheapest” coincide.



Try it yourself: a second graph

Before moving on, trace BFS by hand on this graph: start **P**, goal **T**, edges P–Q, P–R, Q–T, R–T, R–S (all unweighted, alphabetical successor order).

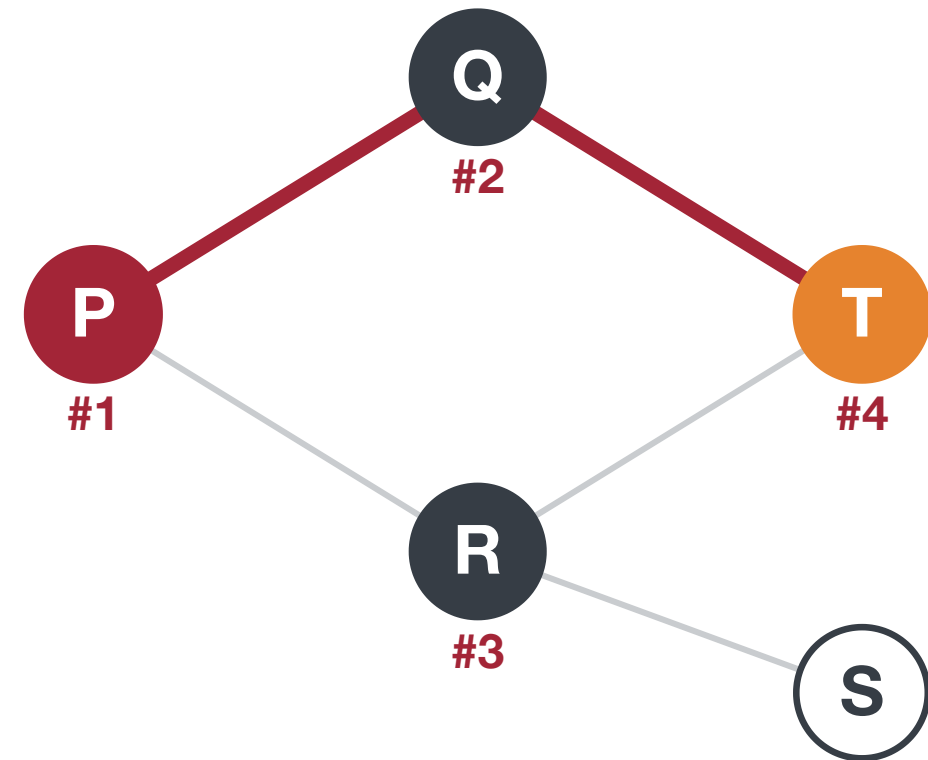
- What order are nodes expanded in?
- Which node is popped when T is first found?
- Do not check the next slide until you have an answer



start P · goal T · all edges unweighted

Try it yourself: the answer

- Frontier trace: $[P] \rightarrow \text{pop } P, \text{ add } Q, R \rightarrow [Q, R] \rightarrow \text{pop } Q, \text{ add } T \rightarrow [R, T] \rightarrow \text{pop } R, \text{ add } S \text{ (T already in frontier, skip)} \rightarrow [T, S] \rightarrow \text{pop } \mathbf{T} \text{ — goal!}$
- Expansion order: P, Q, R, T
- This matches the general template exactly: FIFO queue, skip nodes already seen — if your hand trace disagreed, check which of those two rules you dropped



expansion order: P, Q, R, T

Check your understanding

- All three uninformed algorithms returned the same route on the road-trip graph. Which route, and why?
- Which algorithm would you pick if the state space might be infinite and memory is generous?
- Which algorithm would you pick if memory is scarce and the state space is finite but very deep?

Next: we finally let the algorithm look at the numbers on the roads.



Search with edge costs

When actions don't all cost the same

Why BFS, DFS, and IDS all broke

- All three chose nodes by **how many steps away** they were, never by **how expensive** the steps were
- On the road trip, that produced S–A–G at cost 15, three times in a row
- The true cheapest route, S–B–C–G at cost 8, was never even preferred — B was barely touched
- We need to expand nodes in order of **path cost so far**, not depth



Uniform-cost search (UCS)

Frontier is a **priority queue** ordered by $g(n)$, the cumulative cost from the start.

- Always expand the frontier node with the lowest $g(n)$
- **Complete** and **optimal**, provided all step costs are positive
- This is exactly **Dijkstra's algorithm**, run until a goal is popped



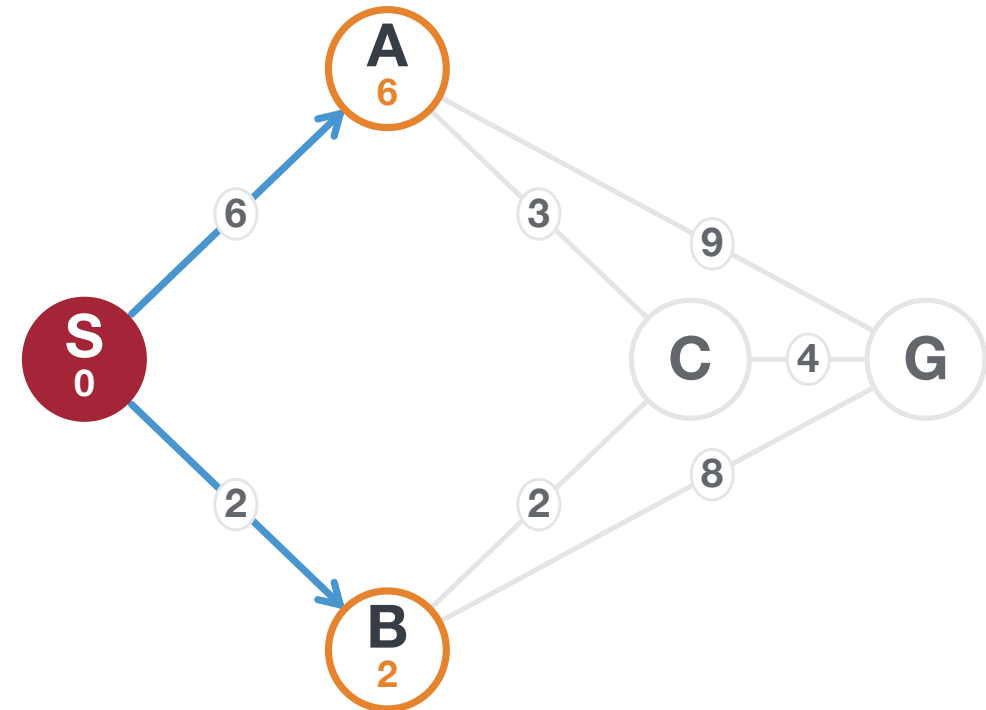
UCS mechanics: relaxation

- On expansion, update a neighbor's g value if a cheaper path to it is just discovered — this update is called **relaxation**
- A neighbor already in the frontier does not need to be re-added; its existing entry's cost and parent are simply updated
- This is the one behavior BFS/DFS/IDS never needed, because they never revised how “good” a node was once queued



UCS trace, step 1

- Frontier (priority queue by g): {S: 0}.
- Pop **S** (g = 0, lowest). Mark explored. Not the goal.
- Expand: A gets $g = 0 + 6 = 6$; B gets $g = 0 + 2 = 2$.
- Frontier: {A: 6, B: 2}



ring shows g: best cost found so far

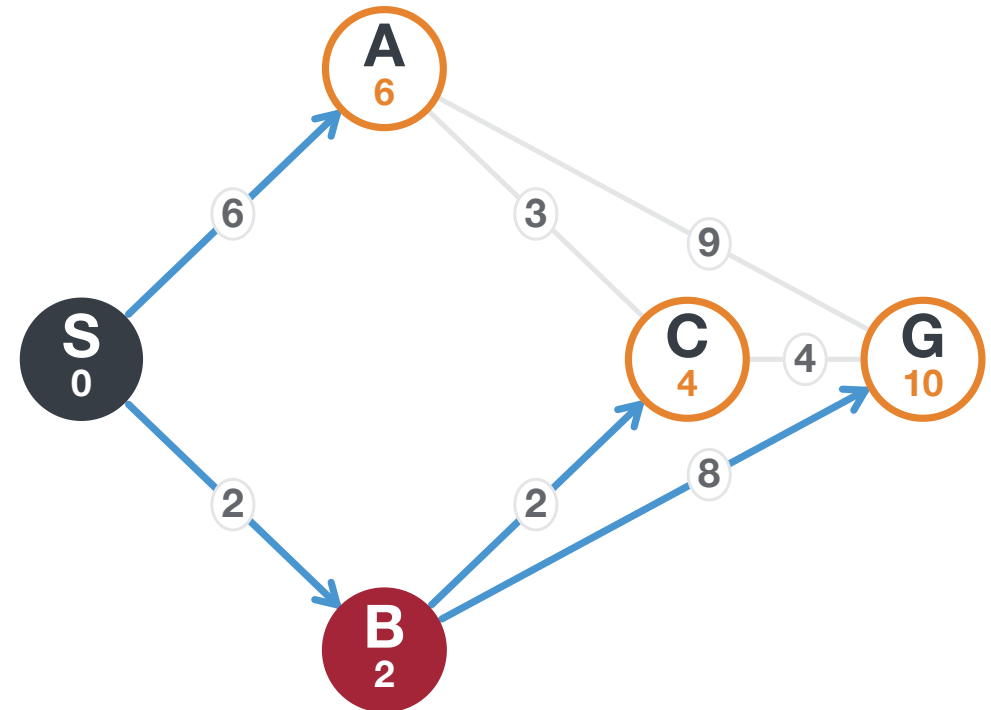
queue by g **B:2** **A:6**

○ unseen ○ frontier ● explored

● popped this step → parent pointer

UCS trace, step 2

- Pop **B** ($g = 2$, lowest). Mark explored (parent of B is S). Not the goal.
- Expand B: S is explored, skip. C gets $g = 2 + 2 = 4$. G gets $g = 2 + 8 = 10$.
- Frontier: {A: 6, C: 4, G: 10}



ring shows g: best cost found so far

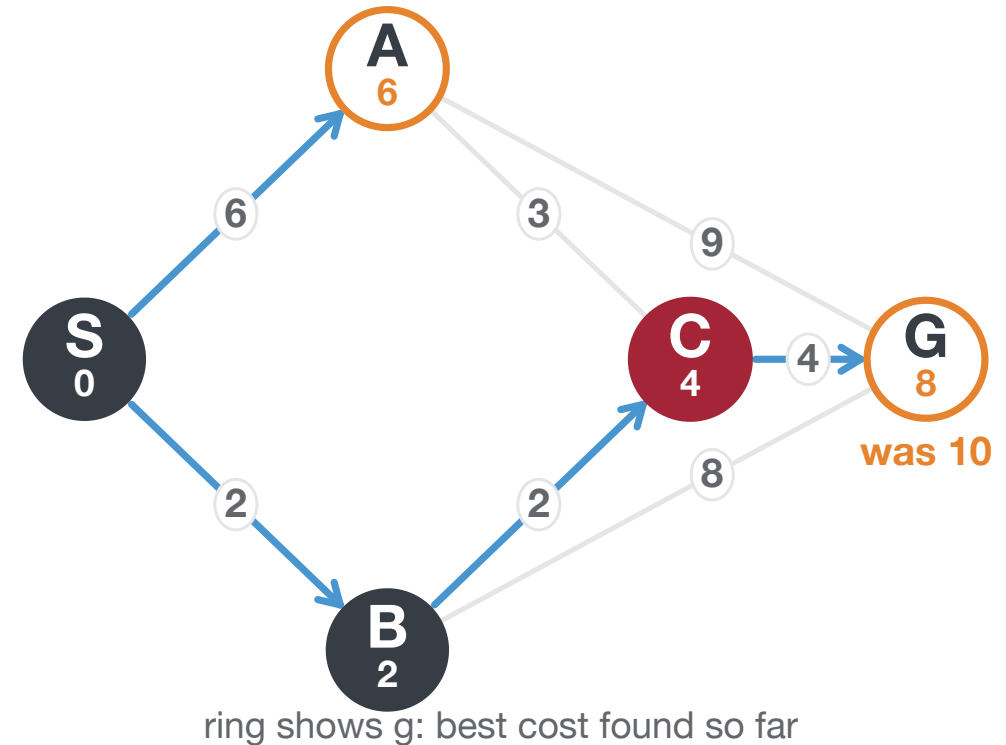
queue by g **C:4** **A:6** **G:10**

○ unseen ○ frontier ● explored

● popped this step → parent pointer

UCS trace, step 3 — relaxation happens

- Pop **C** ($g = 4$, lowest). Mark explored (parent of C is B). Not the goal.
- Expand C: A is not yet explored — via C, A would cost $4 + 3 = 7$, which is *worse* than A's current 6, so **no update**.
- G via C would cost $4 + 4 = 8$, which is *better* than G's current 10 — **relax G's cost to 8, and its parent to C**.
- Frontier: {A: 6, G: 8}

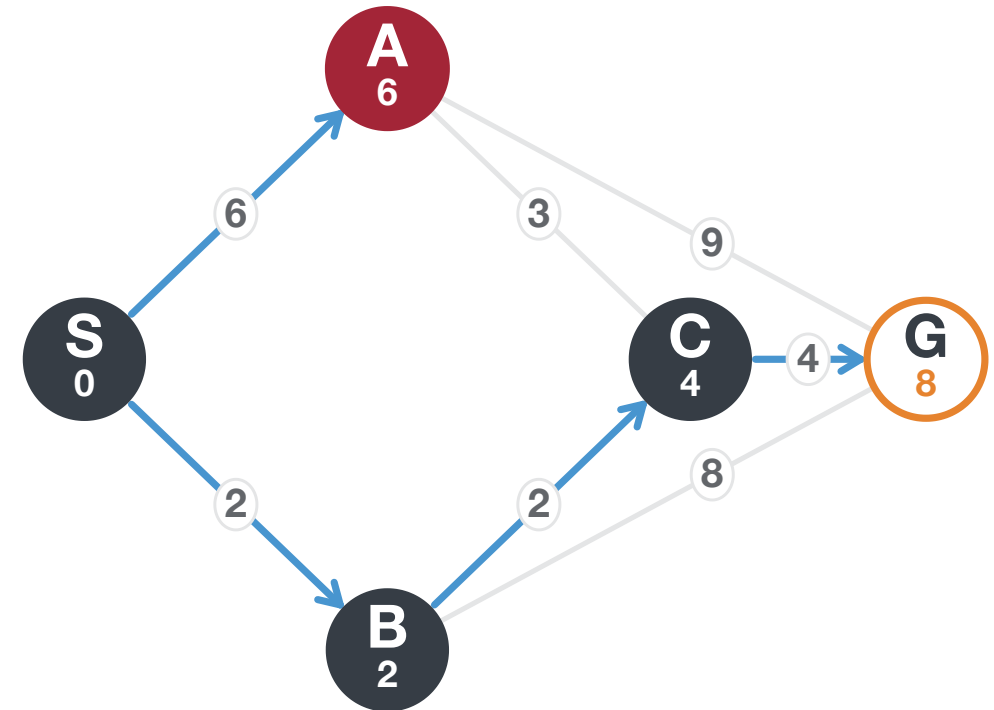


queue by g **A:6** **G:8**

○ unseen ○ frontier ● explored
● popped this step → parent pointer

UCS trace, step 4

- Pop **A** ($g = 6$, lowest). Mark explored (parent of A is S). Not the goal.
- Expand A: S explored, skip. C explored, skip. G via A would cost $6 + 9 = 15$, worse than G's current 8 — no update.
- Frontier: {G: 8}



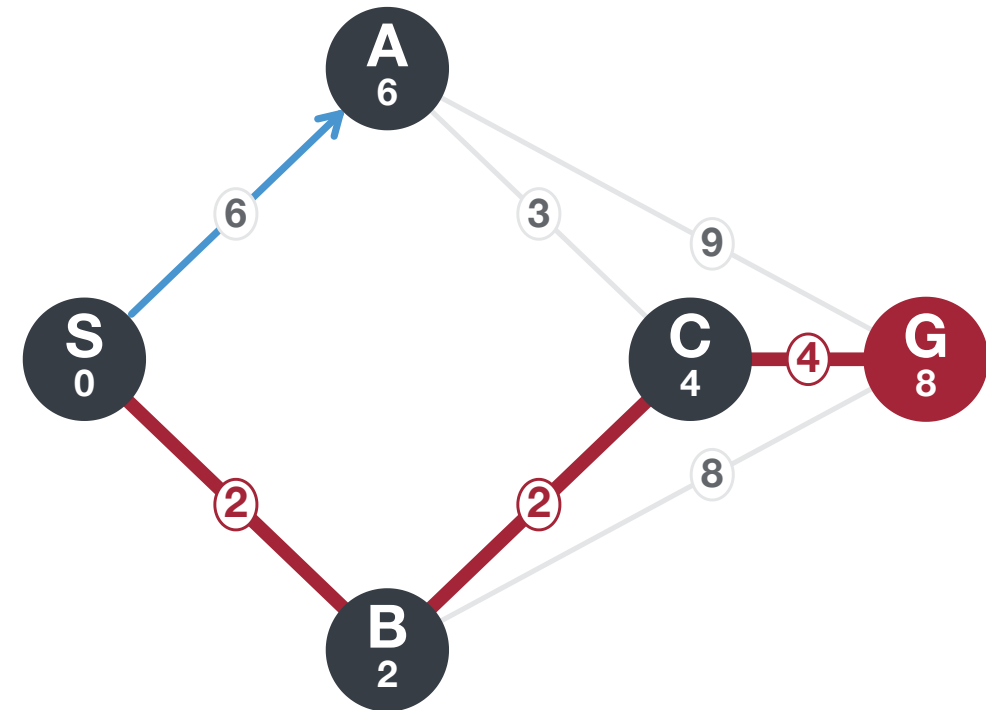
ring shows g: best cost found so far

queue by g **G:8**

○ unseen ○ frontier ● explored
● popped this step → parent pointer

UCS trace, step 5 — done

- Pop **G** ($g = 8$). It is the goal!
- Parent pointers: $G \leftarrow C \leftarrow B \leftarrow S$.
- **UCS's answer: $S \rightarrow B \rightarrow C \rightarrow G$, cost 8** — the true optimum.



ring shows g : best cost found so far

queue by g empty

○ unseen ○ frontier ● explored
● popped this step → parent pointer

UCS full trace, in one table

Step	Popped (g)	Frontier after expansion
1	S (0)	A: 6, B: 2
2	B (2)	A: 6, C: 4, G: 10
3	C (4)	A: 6, G: 8 (relaxed from 10)
4	A (6)	G: 8 (no change; 15 is worse)
5	G (8)	goal

Five nodes expanded, same as BFS — but this time, the cost is right.



UCS: completeness, optimality, cost

- **Complete** and **optimal** whenever every step cost is positive
- Time/space: roughly $O(b^{(1 + C^*/\epsilon)})$, where C^* is the optimal cost and ϵ the minimum step cost — “how many cheap steps deep” the solution effectively is
- Same blind spot as BFS: no notion of *direction* toward the goal — it explores uniformly outward in every direction, regardless of where G actually is

UCS is optimal but blind. The next section fixes the “blind” part.



Implementing the frontier: why the data structure matters

- BFS/DFS need only a plain queue/stack — $O(1)$ push and pop
- UCS and A* need a **priority queue**, typically a binary heap: $O(\log n)$ push and pop-minimum, where n is the current frontier size
- Relaxation (updating an existing entry's priority) needs either a heap that supports decrease-key, or the simpler trick of pushing a duplicate entry and ignoring stale ones when popped — both appear in real implementations, including the one you will write for Assignment 3
- This is where the $O(b^{(1+C^*/\epsilon)})$ bound's hidden constant actually comes from in practice



Where uniform-cost search shows up in practice

- GPS and network routing (this is literally Dijkstra's original 1959 motivating application)
- Currency arbitrage detection, modeled as shortest/longest paths in a graph of exchange rates
- Any setting with real costs and no cheap way to estimate distance to the goal



Check your understanding

- What operation does UCS perform that none of BFS, DFS, or IDS needed? Where did it happen in the trace?
- Why did UCS expand the same five nodes as BFS, yet return a different (and correct) answer?



Informed search: A^*

Adding a sense of direction

Heuristics

A **heuristic** $h(n)$ estimates the cost from node n to the nearest goal — a guess, computed cheaply, without solving the problem.

- Example: straight-line distance as a heuristic for road distance
- Example: Manhattan distance as a heuristic for sliding-tile puzzles
- A good heuristic is *informative* (close to the true cost) and *cheap to compute*



What makes a heuristic good

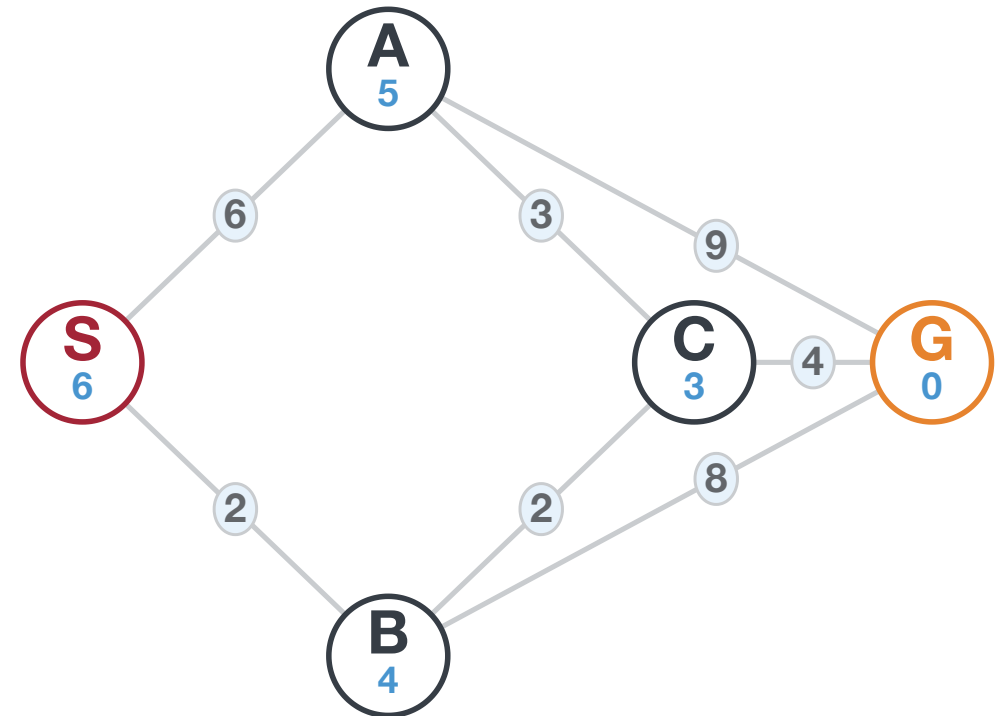
- **Informative:** closer to the true remaining cost means fewer nodes explored
- **Cheap:** computing $h(n)$ should be much cheaper than solving the remaining problem — otherwise you would just solve it
- **Zero effort, zero benefit:** $h(n) = 0$ for every node is always technically valid, and reduces A^* exactly to UCS
- The art of applying A^* to a new problem is almost entirely the art of designing h



Heuristic values for the road trip

We will use each town's straight-line distance to G as h :

Town	$h(n)$
S	6
A	5
B	4
C	3
G	0



blue number in a ring is $h(n)$

These numbers were not picked at random — the next two slides check that they behave.

Admissibility

A heuristic is **admissible** if it never overestimates the true cost to the goal: $h(n) \leq h^*(n)$ for every node n .

- Guarantees A^* will not miss a cheaper solution by trusting an overconfident estimate
- The trivial heuristic $h(n) = 0$ is admissible — it reduces A^* to UCS



Checking admissibility on the road trip

Compare our h against the true cheapest costs computed earlier:

Town	$h(n)$	true cost to G	admissible?
S	6	8	yes, $6 \leq 8$
A	5	7	yes, $5 \leq 7$
B	4	6	yes, $4 \leq 6$
C	3	4	yes, $3 \leq 4$
G	0	0	yes, tight

Every row checks out — this heuristic is admissible.



Consistency

A heuristic is **consistent** (monotone) if, for every edge (n, n') : $h(n) \leq \text{cost}(n, n') + h(n')$

- A form of the triangle inequality applied to the heuristic
- Consistency implies admissibility (not vice versa)
- Consistency guarantees $g(n)$ along the path A^* finds is never revised after expansion — simplifies the optimality argument, and lets A^* skip the relaxation bookkeeping UCS needed after a node is popped



Checking consistency on the road trip

Every edge, both directions:

Edge	$h(n) \leq \text{cost} + h(n')$?
S–A (6)	$6 \leq 6+5=11 \checkmark \cdot 5 \leq 6+6=12 \checkmark$
S–B (2)	$6 \leq 2+4=6 \checkmark \text{ (tight)} \cdot 4 \leq 2+6=8 \checkmark$
A–C (3)	$5 \leq 3+3=6 \checkmark \cdot 3 \leq 3+5=8 \checkmark$
A–G (9)	$5 \leq 9+0=9 \checkmark$
B–C (2)	$4 \leq 2+3=5 \checkmark \cdot 3 \leq 2+4=6 \checkmark$
B–G (8)	$4 \leq 8+0=8 \checkmark$
C–G (4)	$3 \leq 4+0=4 \checkmark$

Every edge checks out — this heuristic is consistent, not merely admissible.



Consistency implies admissibility, not the reverse

- If h is consistent everywhere, summing the triangle inequality along any path to G shows $h(n)$ never exceeds the true remaining cost — so consistent $\Rightarrow$ admissible
- The converse can fail: a heuristic can respect the global “never overestimate” rule while violating it *locally* on a single edge
- Example sketch: shrink S ’s heuristic from 6 down to 1. Still admissible ($1 \leq 8$). A real counterexample needs an edge where the heuristic *drop* between neighbors exceeds that edge’s cost; the point is that admissibility is a global bound and consistency is a per-edge bound, and the second is strictly stronger



A* search

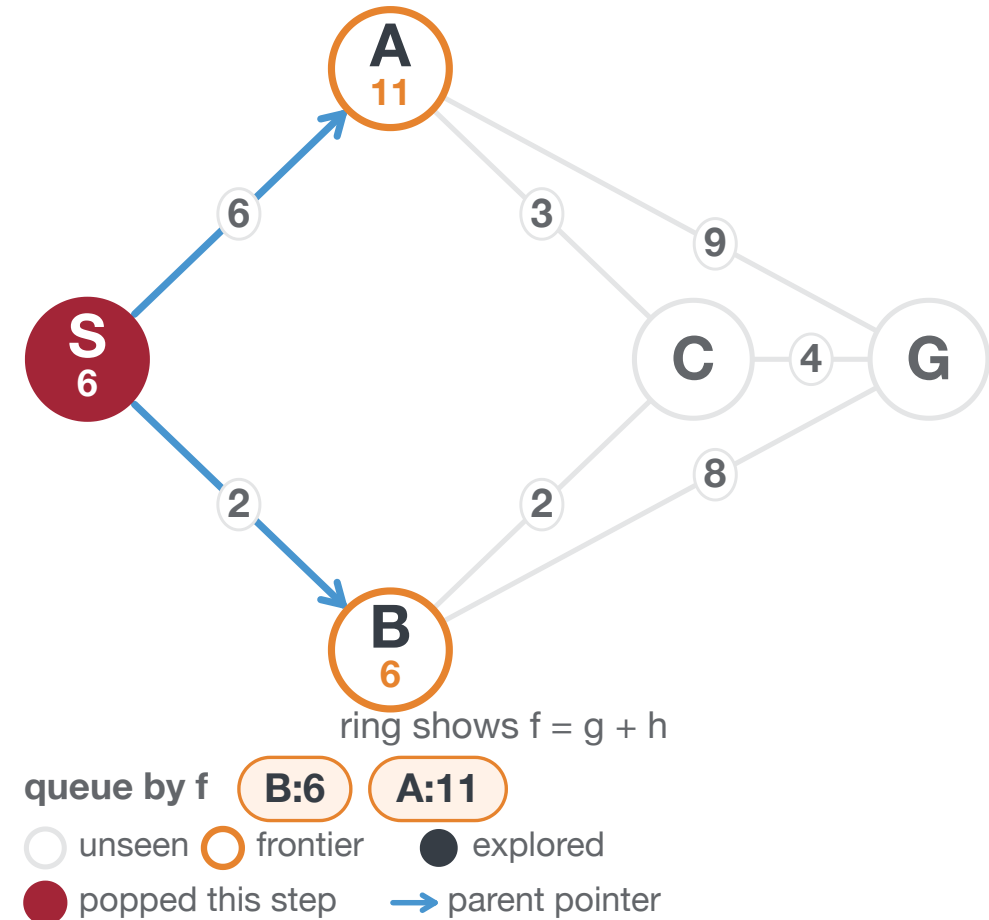
Frontier is a priority queue ordered by $f(n) = g(n) + h(n)$.

- $g(n)$: cost so far (like UCS)
- $h(n)$: estimated cost to go (the new ingredient)
- Expand the node with the lowest *total* estimated cost — cost paid plus cost guessed remaining



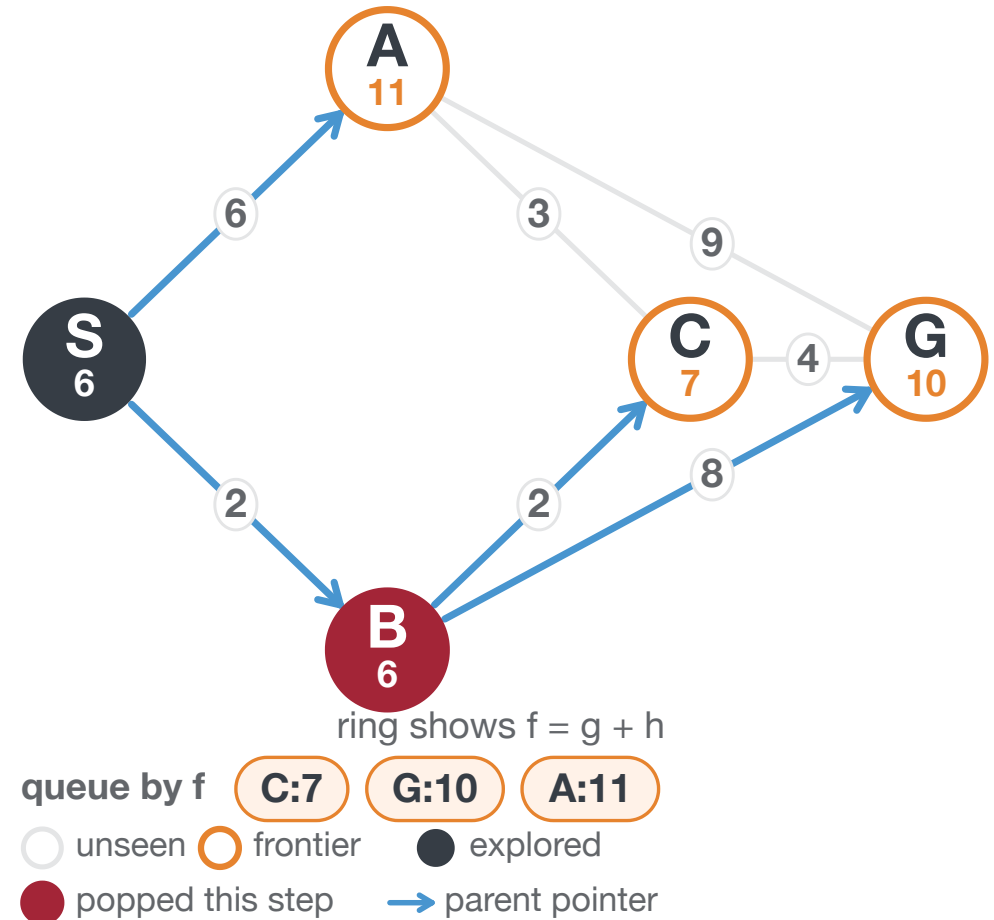
A* trace, step 1

- Frontier (priority queue by $f = g + h$): {S: $g=0$, $h=6$, $f=6$ }
- Pop **S** ($f = 6$). Mark explored. Not the goal.
- Expand: A gets $g=6$, $h=5$, $f=11$. B gets $g=2$, $h=4$, $f=6$.
- Frontier: {A: $f=11$, B: $f=6$ }



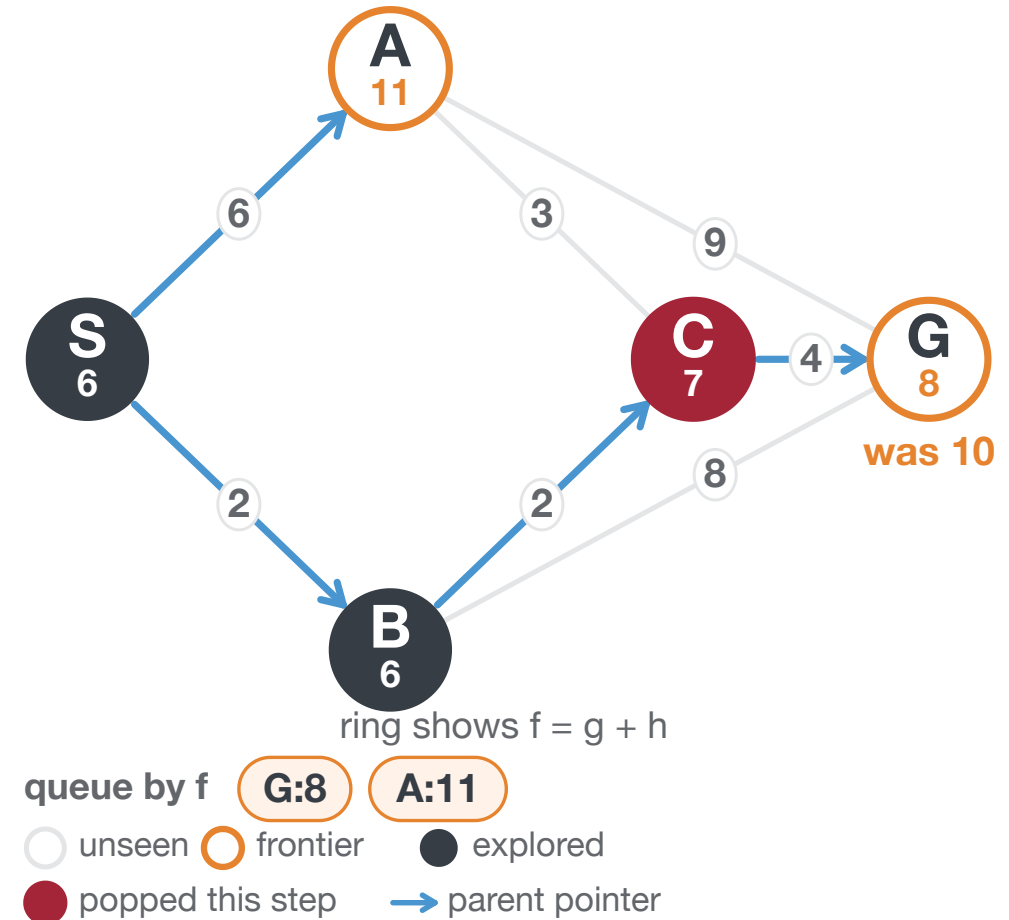
A* trace, step 2

- Pop **B** ($f = 6$, lowest). Mark explored (parent of B is S). Not the goal.
- Expand B: C gets $g=4$, $h=3$, $f=7$. G gets $g=10$, $h=0$, $f=10$.
- Frontier: {A: $f=11$, C: $f=7$, G: $f=10$ }



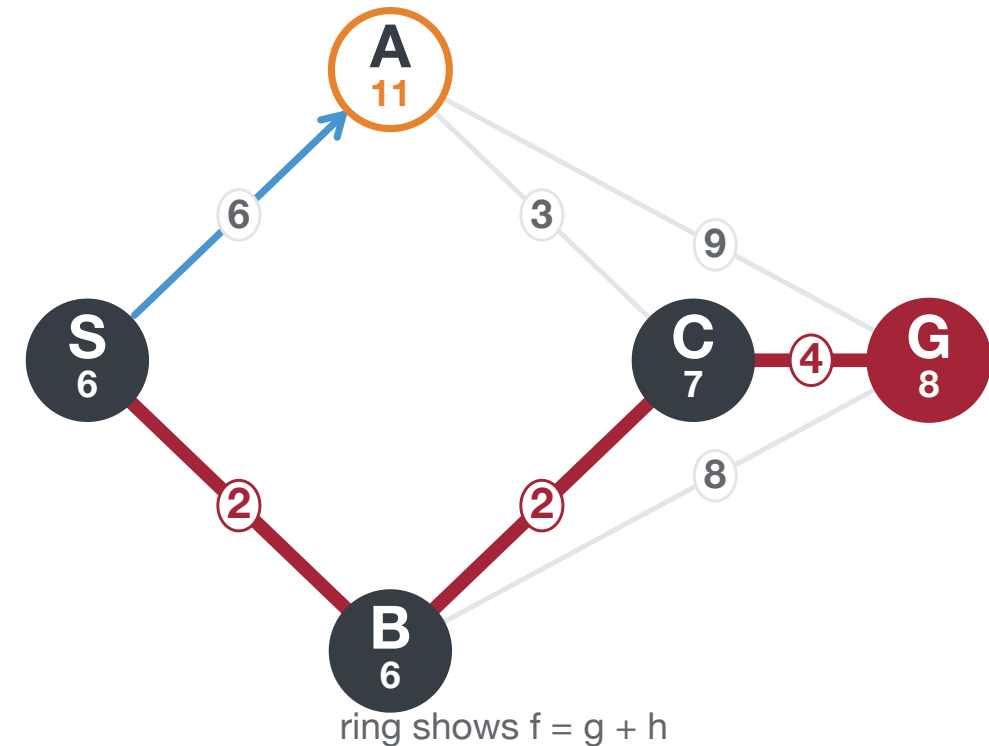
A* trace, step 3 — relaxation, same as UCS

- Pop **C** ($f = 7$, lowest). Mark explored (parent of C is B). Not the goal.
- Expand C: A via C would be $g=7$, worse than A's current $g=6$ — no update. G via C would be $g=8$, $h=0$, $f=8$, better than G's current $f=10$ — **relax G to $f=8$, parent C**.
- Frontier: {A: $f=11$, G: $f=8$ }



A* trace, step 4 — done, and A is skipped

- Pop **G** ($f = 8$, lowest — *lower than A's $f = 11$*). It is the goal!
- **A is never expanded.** A* stops here, without ever popping A.
- Parent pointers: $G \leftarrow C \leftarrow B \leftarrow S$. **A*'s answer: $S \rightarrow B \rightarrow C \rightarrow G$, cost 8** — the true optimum, same as UCS.



queue by f **A:11**

○ unseen ○ frontier ● explored
● popped this step → parent pointer

A* vs. UCS: the payoff, side by side

	UCS	A*
Nodes expanded	S, B, C, A, G (5)	S, B, C, G (4)
A expanded?	yes, at step 4	never
Route found	S–B–C–G	S–B–C–G
Cost found	8 (optimal)	8 (optimal)

Same optimal answer, one fewer node expanded. On a graph with a million nodes, that gap is the entire point of using a heuristic.



Why A* skips A: reading it off the f-values

- When G is popped, its f-value is 8
- A is still sitting in the frontier with $f = 11$ — A* only expands the *lowest-f* node, and $11 > 8$
- UCS had no h to inflate A's priority, so it compared $g(A)=6$ against $g(G)=8$ and expanded A first
- The heuristic is what told A* “A cannot possibly lead anywhere cheaper than what we already have”



A* optimality (proof sketch)

Claim: with an admissible heuristic, the first goal node A^* pops off the frontier has the optimal cost.

- Suppose a suboptimal goal G_2 is popped before the optimal goal G
- Then some node n on the optimal path to G is still in the frontier, with $f(n) \leq f^*(G) < f(G_2)$
- Since h is admissible, $f(n)$ never exceeds the true optimal cost through n
- Contradiction — A^* would have popped n , not G_2 , first

Consistency additionally guarantees this holds even with graph search (repeated states pruned) — exactly the setting we traced above.



Greedy best-first search: the other extreme

- What if we order the frontier purely by $h(n)$, ignoring $g(n)$ entirely?
- This is **greedy best-first search** — fast, but not optimal and not even complete in general
- Spectrum: UCS uses g only, greedy best-first uses h only, **A* uses both** — this is exactly why A* is often described as sitting “between” the two
- On the road trip, greedy best-first from S would immediately prefer B ($h=4$) over A ($h=5$) — correct here, but only by luck; nothing stops it from being fooled by a misleading heuristic elsewhere in a larger graph



Designing heuristics: 8-puzzle setup

- State: a 3×3 grid holding tiles 1–8 and one blank, in some arrangement
- Actions: slide a tile adjacent to the blank into the blank's position
- Goal: a specific target arrangement (e.g., tiles in numeric order)
- Branching factor is small (≤ 4), but the state space is still $9!/2 \approx 181,440$ reachable states — too big to search blindly without a good heuristic



Designing heuristics: 8-puzzle, h_1 (misplaced tiles)

- $h_1(n)$ = number of tiles not in their goal position (blank excluded)
- Cheap to compute: one pass over the board
- Admissible: every misplaced tile needs at least one move to fix, so the true cost is at least h_1
- Weakly informative — ignores *how far* each tile has to travel



Designing heuristics: 8-puzzle, h_2 (Manhattan distance)

- $h_2(n)$ = sum, over all tiles, of the grid distance ($|\Delta\text{row}| + |\Delta\text{col}|$) between a tile's current position and its goal position
- Still admissible: each unit of grid distance needs at least one move
- Strictly more informative than h_1 whenever any tile is more than one square from home — and *more informative, still admissible* means **fewer nodes expanded**, a direct speed win
- This is the standard heuristic used in most textbook 8-puzzle solvers



Designing heuristics: route planning, revisited

- $h(n)$ = straight-line (Euclidean or great-circle) distance from n to the destination
- Admissible whenever no road beats “as the crow flies” — true for ordinary road networks
- This is exactly the h we used on the road-trip graph, just imagined as literal geography instead of a table of numbers



The relaxation recipe for generating admissible heuristics

- General principle: **solve a simplified (relaxed) version of the problem exactly**, and use its cost as h
- 8-puzzle $h1$: relax “slide into an adjacent blank” down to “teleport anywhere” — cost becomes “is it home or not,” i.e., misplaced-tile count
- 8-puzzle $h2$: relax “slide one square, avoiding other tiles” down to “slide through other tiles” — cost becomes Manhattan distance
- Route planning: relax “follow the road network” down to “fly directly” — cost becomes straight-line distance
- Trade-off: a more accurate relaxation is often more expensive to compute — the net win must still be positive



Common pitfall: an inadmissible heuristic breaks optimality

- Suppose we had (wrongly) set $h(A) = 9$ instead of 5 on the road trip
- Then $f(A)$ at the moment A is generated would be $g=6 + h=9 = 15$
- That does not change *this particular* trace's answer, but in general, an *overestimating* heuristic can make A^* pop a worse goal before a better one is ever reconsidered — the optimality proof's key step (*$f(n)$ never exceeds the true cost*) simply stops holding
- Lesson: a heuristic that is merely “a good guess” is not enough — it must be provably $\leq$ the true cost, on every node, or the optimality guarantee is void

State-space size, concretely

- Road-trip graph: 5 states — small enough to trace by hand, which is exactly why we used it
- 8-puzzle: $9!/2 \approx 181,440$ reachable states — already too many to search blindly without a heuristic
- Rubik's Cube: about 4.3×10^{19} states — solvable optimally by machine only because of extremely informative heuristics (pattern databases) combined with A*-style search

The lesson scales: the better the heuristic, the larger the state space you can still search in practice.



Weighted A* and bounded-suboptimal search

- Practical engineering trick: order the frontier by $f(n) = g(n) + w \cdot h(n)$ for some weight $w > 1$
- Trades the optimality guarantee for speed — larger w pushes the search harder toward the goal and expands fewer nodes, at the cost of a provable bound like “at most w times the optimal cost”
- Common in time-constrained settings (real-time pathfinding in games and robotics) where “close to optimal, fast” beats “optimal, slow”



Where informed search shows up in practice

- Game pathfinding (units navigating a map) is close to the textbook A* setting almost unchanged
- Robotics motion planning, often A* over a discretized configuration space
- Any planning problem where a cheap, honest estimate of “how far is left” is available



Check your understanding

- On the road trip, why did A^* expand one fewer node than UCS, and which node was it?
- Is $h(n) = 0$ ever a reasonable choice? What does A^* become if you use it?
- For the 8-puzzle, why is Manhattan distance admissible, and why is it more informative than counting misplaced tiles?



Complexity and choosing an algorithm

Same problem, different frontiers

Bidirectional search

- Idea: run one search forward from the start and another backward from the goal, and stop when their frontiers meet
- Can cut the *effective* depth roughly in half — since work grows exponentially in depth, this is a large saving even though it “only” halves d
- Requires being able to generate predecessors as easily as successors (run actions in reverse) — not always available
- With weighted edges, meeting in the middle does not automatically give the optimal path; extra bookkeeping is needed to guarantee it, which is why it is mentioned here as a tool, not a drop-in replacement for A*



Common pitfalls, collected

- **DFS in an infinite or cyclic space**, run as a *tree* search without a depth bound: never terminates
- **Negative edge weights breaking Dijkstra/UCS**: relaxation assumes a node's optimal cost cannot get cheaper once popped; a negative edge after the pop can violate that (this is exactly why Bellman–Ford exists for graphs with negative weights)
- **An inadmissible heuristic in A***: silently breaks the optimality guarantee, often without any error message — the algorithm still runs, it just may return the wrong answer
- **Forgetting the explored set** in a graph search: correct results, but potentially exponentially slower, or in cyclic graphs, no termination at all



Full complexity comparison

b = branching factor, d = solution depth, m = maximum depth of the tree, C^* = optimal solution cost, ϵ = minimum step cost.

Algorithm	Complete?	Optimal?	Time	Space
BFS	yes	yes (unit cost)	$O(b^d)$	$O(b^d)$
DFS	no	no	$O(b^m)$	$O(bm)$
IDS	yes	yes (unit cost)	$O(b^d)$	$O(bd)$
Dijkstra / UCS	yes	yes	$O(b^{(1+C^*/\epsilon)})$	$O(b^{(1+C^*/\epsilon)})$
A* (admissible h)	yes	yes	problem-dependent, $\leq$ UCS	problem-dependent, $\leq$ UCS



Choosing an algorithm in practice

- All costs equal, shallow solutions likely → **BFS**
- Memory is the binding constraint, any solution acceptable → **DFS**
- Unknown depth, need completeness under a memory budget → **IDS**
- Costs vary, no useful heuristic available → **Dijkstra / UCS**
- Costs vary, and a cheap admissible estimate exists → **A***

A* with a good heuristic is the workhorse of route planning, robotics motion planning, and puzzle solvers.



The whole running example, end to end

Every algorithm's answer on the same road-trip graph (start S, goal G, true optimum 8):

Algorithm	Route found	Cost	Optimal?	Nodes expanded
BFS	S–A–G	15	no	5
DFS	S–A–G	15	no	4
IDS	S–A–G	15	no	4 (at $\ell=2$)
UCS	S–B–C–G	8	yes	5
A*	S–B–C–G	8	yes	4

One table, five algorithms, the whole lecture. The only two that ever looked at edge weights were the only two that got the right answer — and the one that also looked at the goal's *direction* did it with the least work.



Where this goes next

- Next week: **adversarial search** — search where an opponent chooses some of the moves (minimax, game trees, MCTS)
- Everything today assumed a single agent and a static world; games break both assumptions
- **Assignment 3 (Search)** covers BFS, DFS, IDS, Dijkstra, and A* — implement and compare them on a shared benchmark, and watch for the exact pitfalls collected earlier in this section

Bring your search-tree intuition into next week: it is the scaffold minimax is built on.





THANK YOU

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030